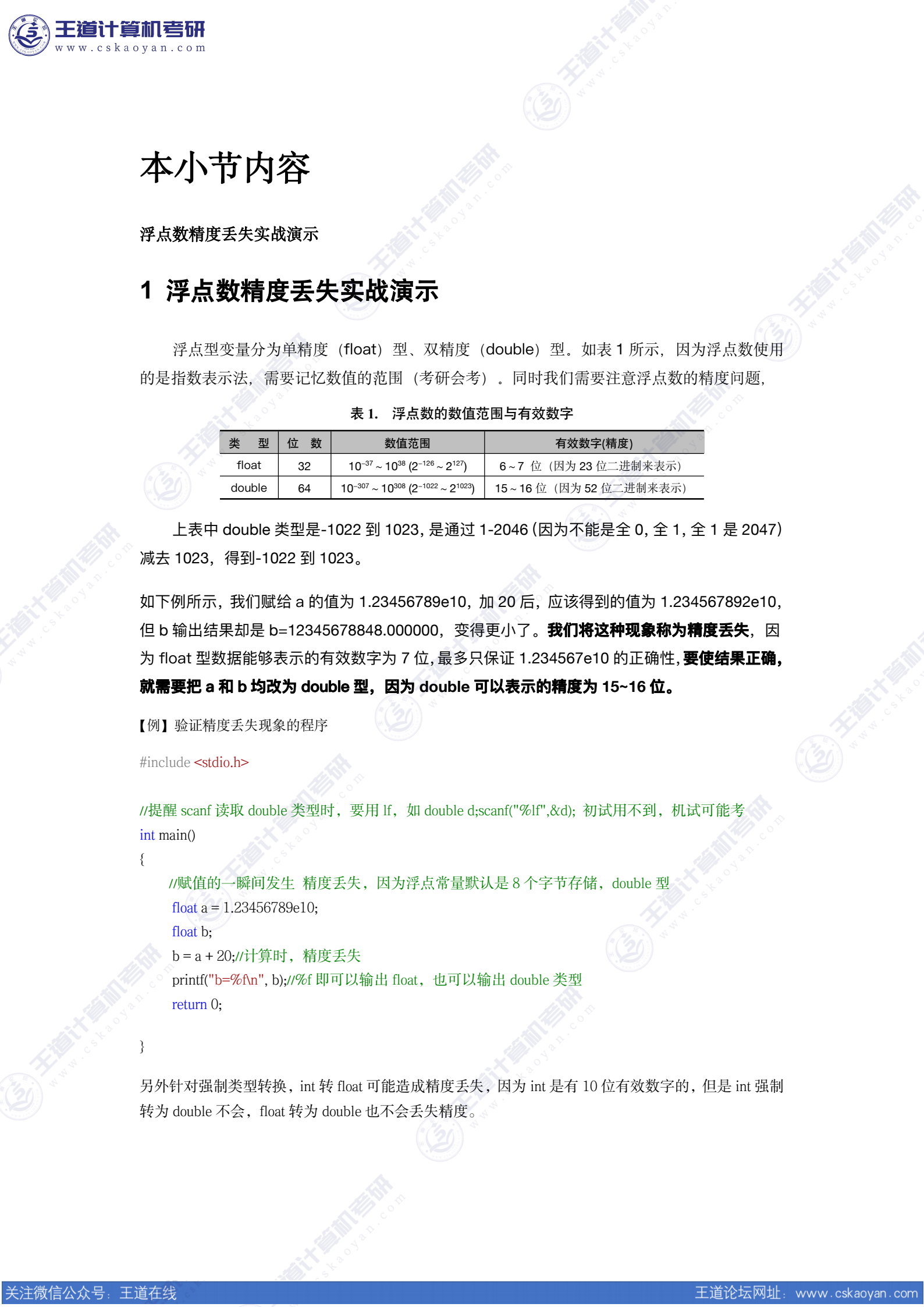




本小节内容

浮点数精度丢失实战演示

1 浮点数精度丢失实战演示

浮点型变量分为单精度 (float) 型、双精度 (double) 型。如表 1 所示，因为浮点数使用的是指数表示法，需要记忆数值的范围（考研会考）。同时我们需要注意浮点数的精度问题，

表 1. 浮点数的数值范围与有效数字

类 型	位 数	数值范围	有效数字(精度)
float	32	$10^{-37} \sim 10^{38}$ ($2^{-126} \sim 2^{127}$)	6 ~ 7 位 (因为 23 位二进制来表示)
double	64	$10^{-307} \sim 10^{308}$ ($2^{-1022} \sim 2^{1023}$)	15 ~ 16 位 (因为 52 位二进制来表示)

上表中 double 类型是-1022 到 1023, 是通过 1-2046 (因为不能是全 0, 全 1, 全 1 是 2047) 减去 1023, 得到-1022 到 1023。

如下例所示，我们赋给 a 的值为 1.23456789e10, 加 20 后，应该得到的值为 1.234567892e10, 但 b 输出结果却是 b=12345678848.000000, 变得更小了。**我们将这种现象称为精度丢失**，因为 float 型数据能够表示的有效数字为 7 位, 最多只保证 1.234567e10 的正确性, **要使结果正确，就需要把 a 和 b 均改为 double 型，因为 double 可以表示的精度为 15~16 位。**

【例】验证精度丢失现象的程序

```
#include <stdio.h>

//提醒 scanf 读取 double 类型时，要用 lf，如 double d;scanf("%lf",&d); 初试用不到，机试可能考
int main()
{
    //赋值的一瞬间发生 精度丢失，因为浮点常量默认是 8 个字节存储，double 型
    float a = 1.23456789e10;
    float b;
    b = a + 20;//计算时，精度丢失
    printf("b=%f\n", b);//%f 即可以输出 float，也可以输出 double 类型
    return 0;
}
```

另外针对强制类型转换，int 转 float 可能造成精度丢失，因为 int 是有 10 位有效数字的，但是 int 强制转为 double 不会，float 转为 double 也不会丢失精度。